

ASLAI

Tradutor de Língua Gestual Americana

2025

Luís Cunha (pg60280), Nuno Araújo (pg61218), Rodrigo Ferreira (pg60392) & Tomás Melo (pg60018)

1 Introdução

Motivação e Contexto



- A barreira linguística entre a comunidade surda e a população ouvinte limita a comunicação fluída e a inclusão social
- A dependência de intérpretes humanos acarreta custos elevados e limita a autonomia dos indivíduos
- Cenários críticos afetados: saúde, educação, serviços públicos

Premissa central:

Hardware simples (câmaras RGB comuns) é suficiente para alimentar um *pipeline* de reconhecimento gestual robusto, desde que suportado por uma camada de processamento inteligente.

O Projeto ASLAI



ASLAI — American Sign Language Artificial Intelligence

Três linhas de desenvolvimento interligadas:

Arquitetura distribuída e modular

Separação clara entre captura, processamento e apresentação

Sensores virtuais

Extração de informação anatómica de elevada precisão a partir de imagens via visão computacional

Modelos de ML

Classificação de gestos estáticos (alfabeto manual) e gestos dinâmicos (palavras e frases)

De LGP para ASL: O Desafio dos Dados



A implementação prática utilizou **American Sign Language (ASL)** em vez de LGP por duas razões:

Disponibilidade de dados

Datasets extensos, bem anotados e de acesso público (Kaggle)

Arquitetura agnóstica

Os princípios de extração de *landmarks*, normalização e classificação são diretamente transferíveis para LGP

“Basta substituir os dados de treino quando um dataset adequado de LGP estiver disponível.”

2 Sensorização e Coleta de Dados

Sensor Primário: Câmara RGB



Afastamento de hardware especializado (luvas com sensores de flexão):

- Barreiras de custo, disponibilidade e conforto que contradizem o objetivo de acessibilidade

Sensor adotado: Câmara RGB integrada em computadores portáteis ou dispositivos móveis

Perturbações externas a mitigar:

- Variações de iluminação
- Fundos complexos
- Oclusão parcial das mãos
- Variabilidade inter-individual

Sensores Virtuais com MediaPipe



A *framework* **MediaPipe** (Google) atua como camada intermediária entre o sensor físico e os modelos:

Por *frame* capturado

Deteção e rastreamento via redes convolucionais profundas

21 pontos chave por mão

Articulações, pontas dos dedos, base do pulso — coordenadas 3D normalizadas

33 pontos do esqueleto corporal

Ombros, cotovelos, pulsos

Vantagem: Em vez de classificar píxeis brutos, os modelos recebem representações estruturadas e invariantes a perturbações visuais.

Datasets Utilizados



Fase de treino:

ASL Alphabet & ASL-HG (Estático)

3000 + 1000 imagens brutas de mãos para treino do classificador do alfabeto manual (Kaggle / Mendeley)

Google Isolated Sign Language Recognition (Dinâmico)

Sequências de *landmarks* já extraídos de vídeos — competição PopSign (Kaggle). Permite treinar modelos de série temporal sem reprocessar vídeo bruto.

Fase de produção (tempo real):

- Câmara do dispositivo acedida via API do browser
- Amostragem controlada: *frames* individuais (estático) ou sequências (dinâmico)
- Envio assíncrono em Base64 para o servidor via REST API

3 Tratamento de Dados

Tratamento para Gestos Estáticos



Cada *frame* gera um vetor de **84 features**:

- **63 coordenadas normalizadas** — (x, y, z) dos 21 *landmarks*, centradas no pulso e normalizadas pela escala da mão
- **21 distâncias euclidianas** — de cada *landmark* ao pulso, normalizadas

Invariâncias aplicadas:

Translação

O *landmark* do pulso é forçado para a origem $(0, 0, 0)$; todas as restantes coordenadas passam a ser relativas

Escala

A distância média entre juntas-chave da mão é usada como fator de normalização, tornando o vetor independente do tamanho físico da mão

Tratamento para Gestos Dinâmicos



Tensor de forma **64 × 225** por sequência:

- **63 valores** — *landmarks* da mão esquerda
- **63 valores** — *landmarks* da mão direita
- **99 valores** — pontos do esqueleto corporal (ombros, cotovelos, pulsos)

Técnicas aplicadas:

Zero-padding / truncagem

Uniformização para janela de 64 *frames*

Normalização por sequência

Subtração da posição média global — isola a trajetória relativa

Speed perturbation

Reamostragem temporal $\pm 25\%$ com probabilidade 50% — robustez a variações de ritmo

4 Modelos de Machine Learning

Modelo Estático — MLP



Objetivo: Reconhecimento do alfabeto manual (letras A–Z, *del*, *space*) — 28 classes

Divisão dos dados: 70% treino · 15% validação · 15% teste

Arquitetura:

Input	84 neurónios
Camadas ocultas	3 camadas densas: 128 → 64 → 32 neurónios (ReLU)
Output	28 neurónios com Softmax
Otimizador	Adam ($lr = 10^{-3}$) com Early Stopping

Curvas de Treino e Matriz de Confusão — MLP

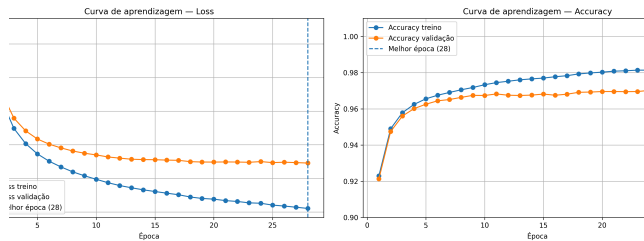


Figure 1: Curvas de aprendizagem

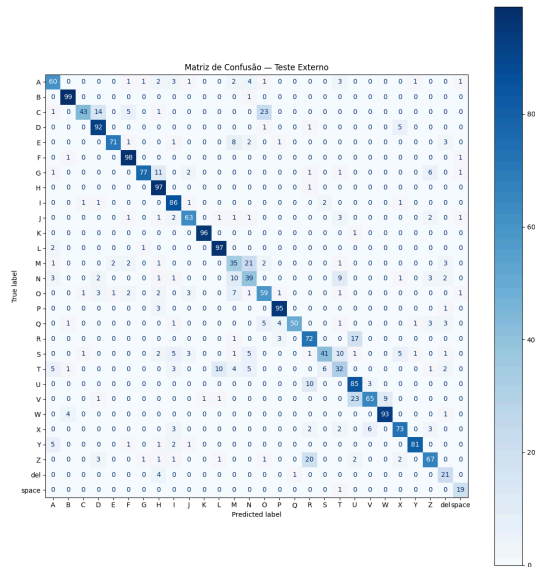


Figure 2: Matriz de confusão

Modelo Dinâmico — Transformer Encoder v2



Arquitetura:

- **Projeção + Positional Encoding:** $225 \rightarrow d_{\text{model}} = 256$ (sinusoidal, $\text{max_len} = 512$)
- **Token [CLS] learnable:** Agregador global da dinâmica temporal
- **6 camadas Transformer:** 4 cabeças de atenção, feedforward = 1024, GELU, Dropout 0.3, norm_first
- **Classificador final:** $\text{Linear}(256, 256) \rightarrow \text{GELU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(256, 100) + \text{Label Smoothing } 0.1$

Resultados:

Modelo	Accuracy Val.	Top-5 Accuracy
Transformer v2 (50 classes)	78.31%	90.59%
Transformer v2 (100 classes)	76.03%	89.72%

Curvas de Treino – Modelos Dinâmicos

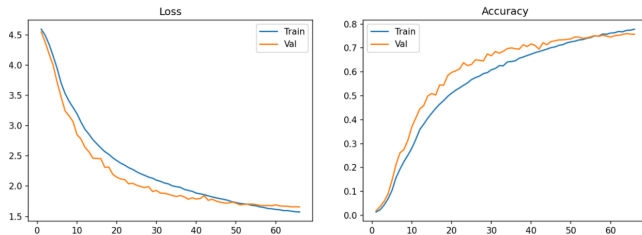


Figure 3: Curvas de aprendizagem

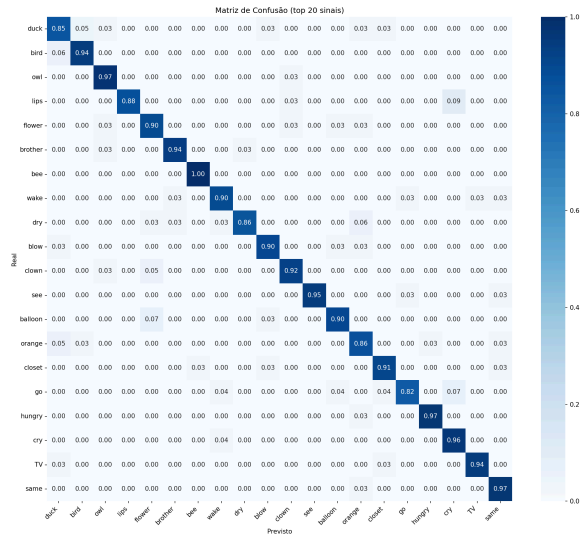


Figure 4: Matriz de confusão - top 20 gestos

5 Arquitetura do Sistema

Visão Geral – Sistema de Microserviços

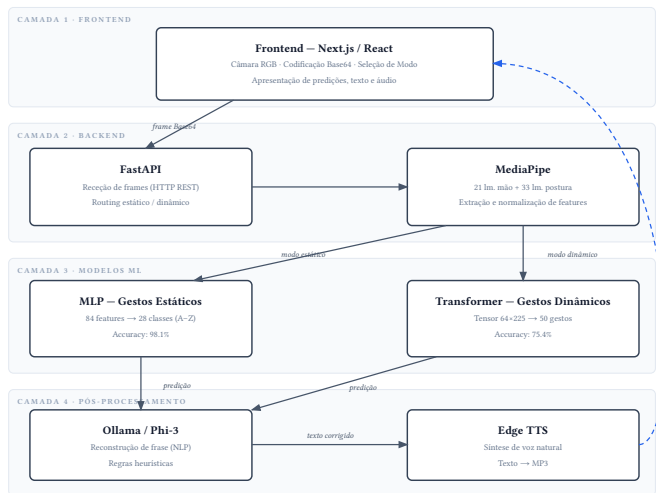


Figure 5: Diagrama da arquitetura do sistema ASLAI

Camada de Apresentação — Frontend



Next.js / React + Tailwind CSS — Single Page Application

Responsabilidades:

- Acesso à câmara do dispositivo via API do browser
- Seleção do modo de operação (estático / dinâmico)
- Captura, espelhamento e codificação dos *frames* em Base64
- Despacho assíncrono de *requests* HTTP para o backend
- Apresentação de predições, frases corrigidas e áudio sintetizado

Camadas de Processamento



Backend — FastAPI (Python)

Recebe imagens, invoca MediaPipe para extração de *landmarks*, e encaminha *features* normalizadas para os modelos. Instâncias MediaPipe pré-carregadas em memória com *threading locks* para concorrência segura.

Pós-processamento Linguístico — Ollama + Phi-3

Motor de inferência local que recebe sequências de palavras brutas e reconstrói frases gramaticalmente corretas e fluídas. Complementado por regras heurísticas para expressões frequentes.

Síntese de Voz — Edge TTS

Converte o texto corrigido em áudio para reprodução pelo frontend. *Endpoint* independente para desacoplamento e flexibilidade futura.

6 Resultados

Desempenho Temporal



Pipeline preditivo completo: captura → envio Base64 → extração de *landmarks* → inferência

Componente	Latência
Inferência MLP (estático)	< 2 ms
Inferência Transformer (dinâmico)	< 8 ms
Transmissão Base64 via HTTP	30 ms – 150 ms (variável com a rede)

O principal fator limitador da latência global é a **transmissão de imagem Base64 via HTTP**, não os modelos em si.

Conclusão



O ASLAI demonstra a viabilidade técnica de um tradutor de língua gestual acessível, modular e adaptável — usando apenas hardware comum.

Conquistas principais:

- MLP estático com **98%+ de accuracy** e generalização total
- Transformer dinâmico com **76–78% de accuracy** para 50–100 gestos
- Arquitetura cliente-servidor modular com latência adequada para tempo real
- Pipeline completo: câmara → gesto → texto → voz